@aman_views

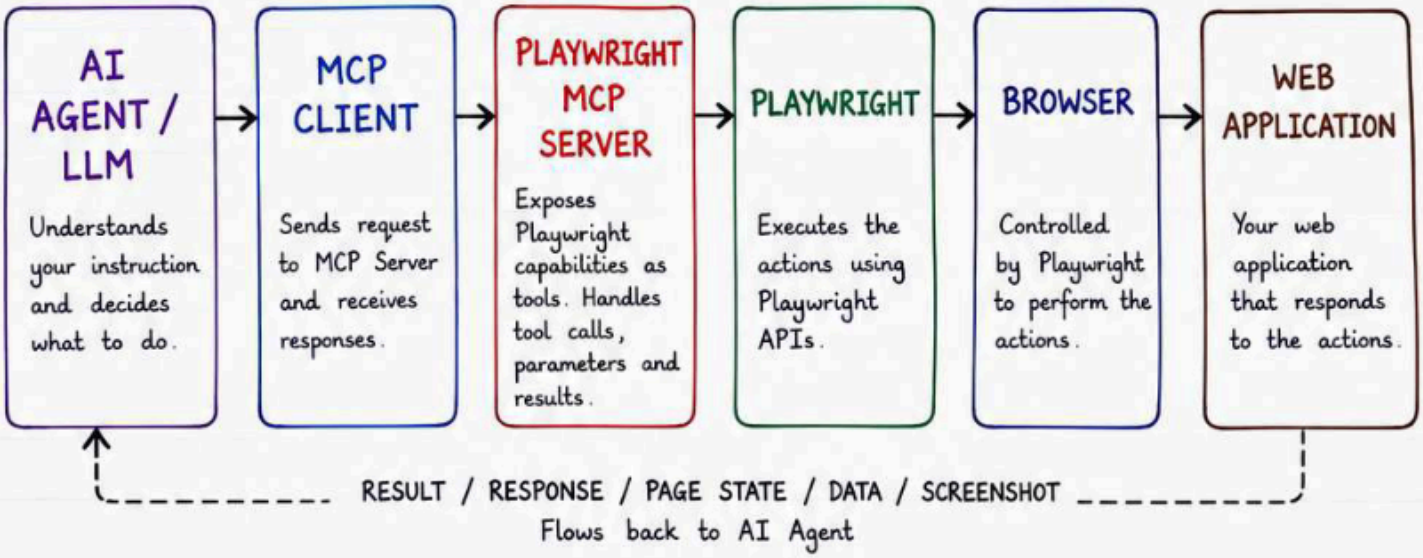
PLAYWRIGHT MCP FOR AUTOMATION TESTING

COMPLETE END-TO-END AI AUTOMATION FLOW

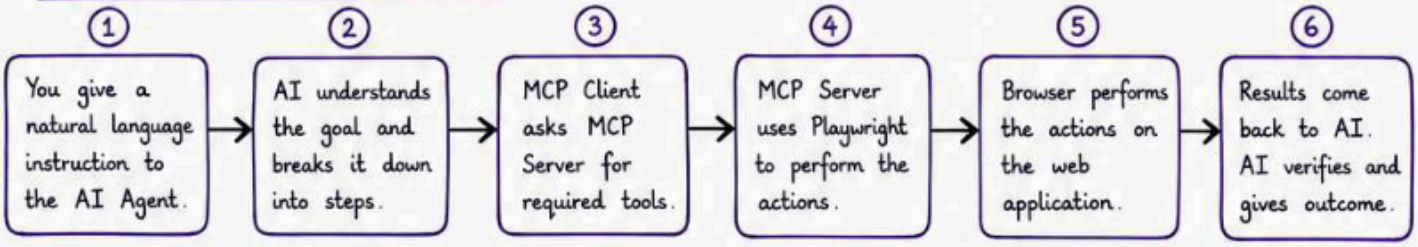
What is Playwright MCP?

Playwright MCP (Model Context Protocol) is a bridge between AI Agents (powered by LLMs) and the browser automation world. It allows AI to understand your instructions, decide what to do, and perform real actions in the browser using Playwright.

HIGH-LEVEL ARCHITECTURE



HOW IT WORKS – SIMPLE FLOW



REAL-WORLD EXAMPLE



KEY TAKEAWAY : Playwright MCP empowers AI Agents to think, plan and act in real browsers. It connects the power of LLMs with the reliability of Playwright.

@aman_views

2. LLM & GENAI FUNDAMENTALS FOR TEST AUTOMATION

→ What is Generative AI ?

Generative AI (GenAI) models can understand human instructions and generate useful outputs like text, code, plans and decisions.

→ What is an LLM ?

LLM (Large Language Model) is a type of GenAI model trained on huge amounts of text data to understand and generate human-like responses.

HOW AN LLM WORKS (Simple View)

INPUT (PROMPT) — You give a question or instruction.

→

TOKENIZATION — Text is broken into small pieces called tokens.

→

MODEL PROCESSING — LLM analyzes the tokens using patterns learned during training (context + knowledge).

→

GENERATE RESPONSE — LLM generates the best possible output token by token.

→

OUTPUT — Final response is returned to the user.

IMPORTANT TERMS

- **Token** : Small part of text (word / sub-word / character).
- **Context** : The information provided to the model (prompt + history).
- **Model** : The brain (LLM) that understands and generates.
- **Response** : The final answer generated by the model.

EXAMPLE

Prompt (Input) : "Open the application, login with user1 and verify dashboard."

↓

LLM Thinks → Understands the goal, breaks it into steps.

Response (Output) → Step-by-step plan with actions to perform.

LLM vs TRADITIONAL AUTOMATION

Aspect	Traditional Automation	LLM + AI Automation
How it works	Hard-coded steps by developer	Natural language instruction understood by AI
Flexibility	Low - changes need code updates	High - AI adapts and plans steps
Maintenance	High - locators & scripts break	Lower - AI can self-heal and adjust
Test Creation	Manual effort	AI can generate tests from requirements
Debugging	Manual log/traces analysis	AI analyzes failure & suggests fixes
Execution	Deterministic (same steps)	Adaptive (AI decides steps at runtime)
Learning	Cannot learn	Learns from context and feedback

WHY LLM ALONE CANNOT CONTROL A BROWSER ?

- LLMs are models - they can think, plan and respond.
- They cannot directly click, type or navigate in a browser.
- They need external tools to take real actions.
- This is where MCP + Playwright comes in.

WHERE MCP FITS IN ?

LLM / AI AGENT — Understands your goal and creates a plan.

→

MCP (BRIDGE) — Provides tools and connects AI with real capabilities.

→

PLAYWRIGHT — Executes actions inside the browser using its APIs.

→

BROWSER — Performs the action on the web page.

→

RESULT BACK TO AI — Outcome / page state / data is returned and AI decides next step.

Continuous loop until the goal is achieved

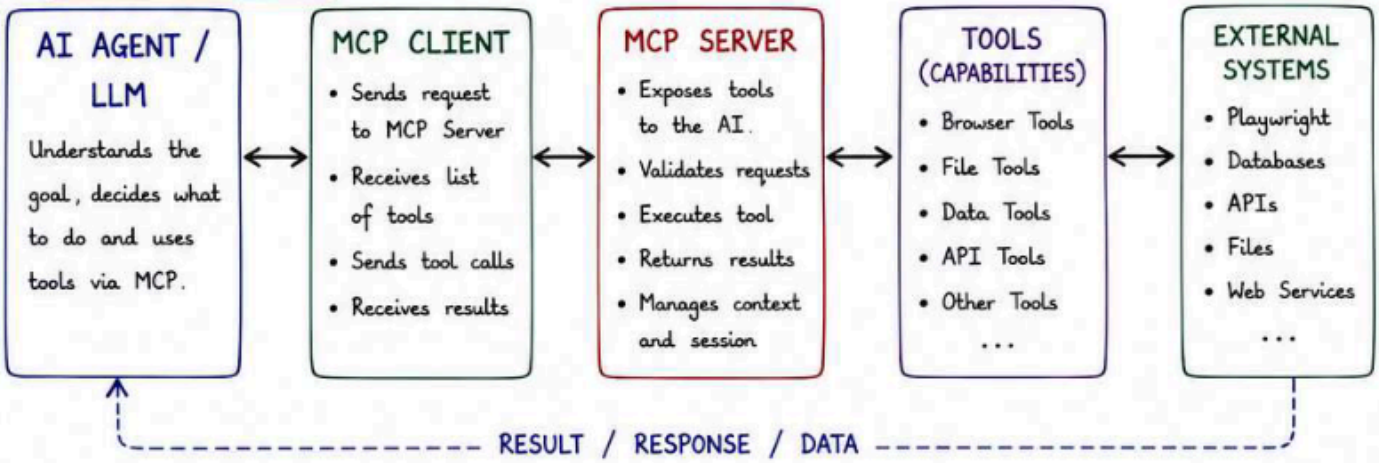
KEY TAKEAWAY: LLM understands instructions and plans the steps. MCP provides the tools. Playwright performs the actions. Together, they enable powerful AI-driven test automation.

@aman_views

3. WHAT IS MCP ? MODEL CONTEXT PROTOCOL

MCP (Model Context Protocol) is an open protocol that allows AI models to discover, access and use external tools, data and capabilities in a safe and standard way.

1. MCP ARCHITECTURE



2. HOW MCP WORKS ? (FLOW)

1. AI AGENT has a goal / instruction from user.
2. MCP CLIENT connects to MCP SERVER.
3. MCP SERVER provides list of available tools.
4. AI decides which tool to use and sends a tool call.
5. MCP SERVER executes the tool with required parameters.
6. Result is returned back to AI Agent.
7. AI uses the result, decides next step or completes the task.

3. KEY CONCEPTS

- → MCP Client : Lives inside the AI application.
- → MCP Server : Exposes tools and handles requests.
- → Tools : Capabilities that AI can call (e.g., browser actions, APIs).
- → Resources : Data that can be read by AI.
- → Prompts : Reusable instructions/templates.
- → Context : Conversation state and tool history.
- → Tool Call : AI asks server to run a tool.
- → Response : Result returned from the tool.

4. MCP vs API (DIFFERENCE)

ASPECT	MCP	API
Purpose	Connect AI to tools in a standard way	Connect systems for data exchange
Discovery	AI can discover available tools	Endpoints are fixed (no discovery)
Protocol	Standard protocol for AI + tools	Custom / fixed per service
Flexibility	AI can choose tools dynamically	Requires manual coding and mapping
Use Case	AI-driven automation	System integration

5. WHY MCP IS IMPORTANT ?

- ✓ Standard way for AI to use external tools.
- ✓ AI can discover and use tools dynamically.
- ✓ Works with different servers and capabilities.
- ✓ Secure and controlled access to tools.
- ✓ Enables powerful AI automation (like Playwright, APIs, DB, Files, etc.).
- ✓ Future-proof for AI Agents and applications.

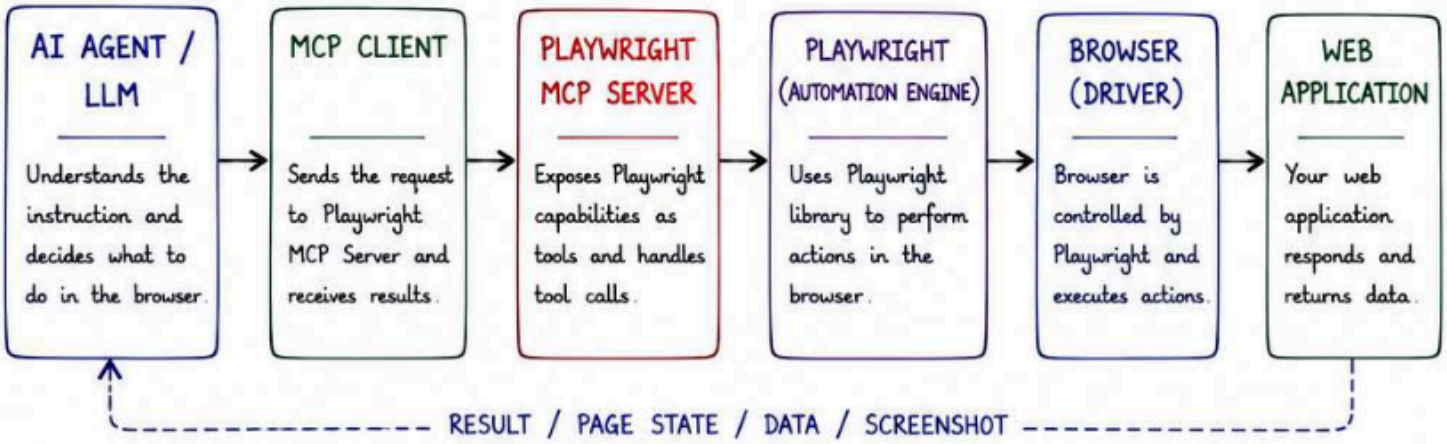
KEY TAKEAWAY : MCP is the bridge that lets AI models use real-world tools and capabilities in a structured, secure and standard way.

@aman_views

4. PLAYWRIGHT MCP - DEEP DIVE

Playwright MCP is an MCP server that exposes all Playwright capabilities as tools so AI Agents can control a real browser through natural language.

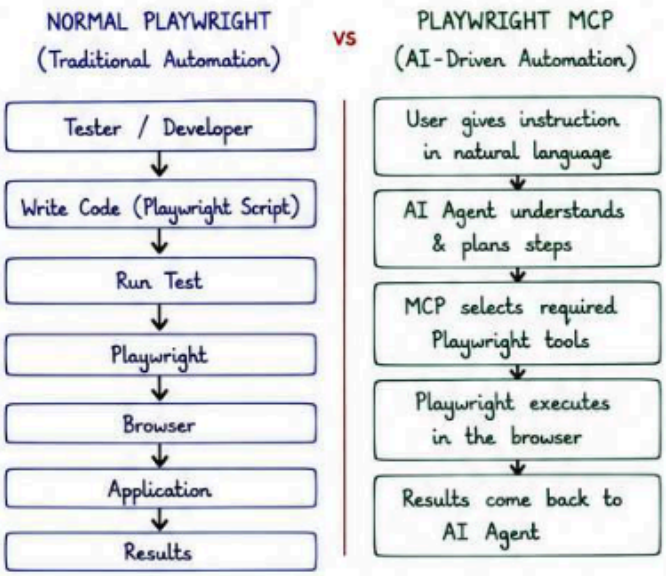
1. HOW PLAYWRIGHT MCP WORKS



2. PLAYWRIGHT CAPABILITIES EXPOSED THROUGH MCP (TOOLS)

NAVIGATE PAGES	LOCATE ELEMENTS	INTERACT	READ DATA	WAIT & RUNTIME	BROWSER CONTROL	UTILITIES	STATE & CONTEXT
Open URLs, reload, go back/ forward.	Find elements using text, role, label, CSS, XPath, etc.	Click, double click, type, select, check, uncheck, hover, drag.	Get text, attribute, innerHTML, value, etc.	Wait for elements, network, load state, timeout.	New page, new tab, close tab, switch, etc.	Screenshot, console logs, downloads, uploads.	Cookies, storage, viewport, user agent.

3. PLAYWRIGHT MCP vs NORMAL PLAYWRIGHT



4. HOW AI DECIDES WHICH ACTION TO TAKE ?

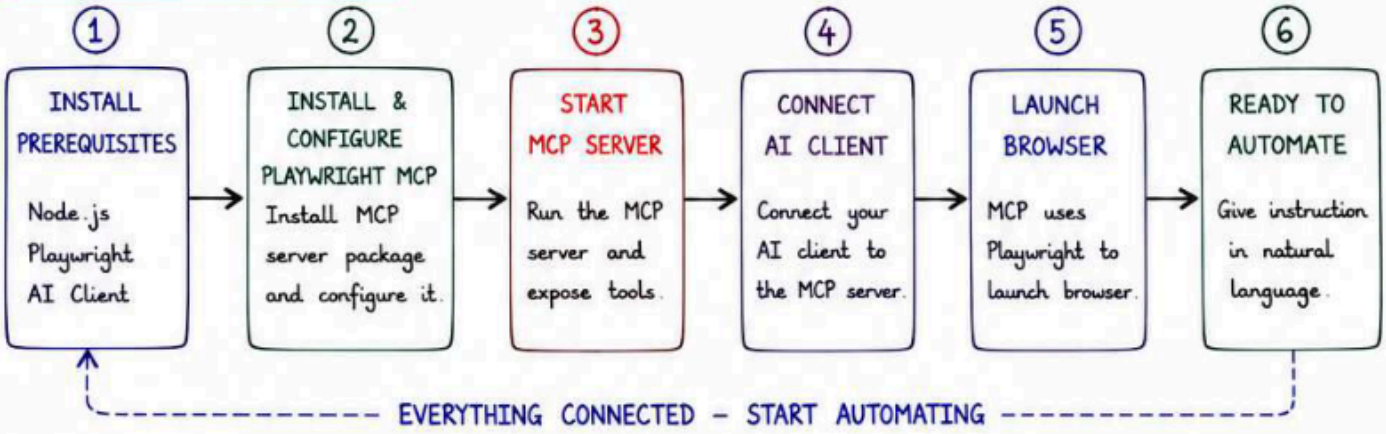
1. AI reads the instruction and current page state.
2. AI understands the goal and breaks it into steps.
3. AI looks at available tools from Playwright MCP.
4. AI selects the best tool(s) for the next step.
5. MCP executes the tool using Playwright.
6. Result / page state / data is returned to AI.
7. AI validates, decides next step or completes the task.

KEY TAKEAWAY: Playwright MCP turns Playwright into an AI-accessible toolkit. AI understands the goal, MCP provides the tools, Playwright performs the actions, and the browser executes them in real time.

5. SETTING UP PLAYWRIGHT MCP

This page covers the complete setup to connect Playwright MCP with your AI client and start automating the browser.

SETUP FLOW (HIGH LEVEL)



1. PREREQUISITES

- Node.js : v18+
- Playwright : Latest Version
- AI Client : Any MCP compatible client (e.g., Claude Desktop, VS Code with MCP extension, etc.)
- Internet : Required for AI model access

2. INSTALL PLAYWRIGHT & MCP SERVER

```
# Install Playwright
npm init playwright@latest

# Install Playwright MCP server
npm install -g @playwright/mcp-server

# (or) add in package.json
"@playwright/mcp-server": "latest"
```

3. MCP SERVER CONFIGURATION (mcp.config.json)

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp-server"],
      "env": {
        "PW_HEADLESS": "false",
        "PW_BROWSER": "chromium"
      }
    }
  }
}
```

4. AI CLIENT CONNECTION EXAMPLE

Example: Claude Desktop (config file)

- Open Claude Desktop settings.
- Go to Developer > MCP Servers.
- Add new server with above configuration.
- Save and restart the client.
- You should see "Playwright" server connected.

5. VERIFY SETUP

- ✓ Start the MCP server.
- ✓ Open AI client.
- ✓ Ask: "List available tools" or "Open a new page".
- ✓ If tools are listed and browser opens, setup is successful!

6. COMMON SETUP ISSUES

- MCP server not starting → Check Node.js version
- Client not connecting → Verify config path & format
- Browser not launching → Check Playwright install
- Tools not listed → Restart MCP server & client
- Port already in use → Change port in config (if needed)

✳ KEY TAKEAWAY: Once MCP server is running and connected to AI client, you can control the browser using natural language through Playwright capabilities.

@aman_views

6. YOUR FIRST AI-DRIVEN PLAYWRIGHT AUTOMATION

In this page we will create a simple end-to-end automation using Playwright MCP with help of an AI Agent using natural language instruction.

1. HOW IT WORKS (HIGH LEVEL FLOW)

1. USER GIVES INSTRUCTION (NATURAL LANGUAGE)
"Open the application, login with test credentials, search for a product and verify the result."

→

2. AI AGENT UNDERSTANDS & PLANS STEPS
- Open URL
- Enter Username
- Enter Password
- Click Login
- Search Product
- Verify Result

→

3. MCP CLIENT SELECTS TOOLS
Chooses required Playwright MCP tools for each step and sends tool calls.

→

4. PLAYWRIGHT MCP EXECUTES ACTIONS
Uses Playwright under the hood to perform the actions in the browser.

→

5. BROWSER PERFORMS ACTIONS
Browser navigates, clicks, types and loads pages as per Playwright commands.

→

6. RESULT RETURNED TO AI
Page state, data and screenshots are returned. AI validates and gives final result.

RESPONSE / PAGE STATE / DATA / SCREENSHOT

2. EXAMPLE: LOGIN → SEARCH → VERIFY

1. Navigate to https://demowebshop.tricentis.com
2. Enter Username: testuser
3. Enter Password: Test@123
4. Click on Login button
5. In search box, type: laptop
6. Press Search / Click Search button
7. Verify that search results page is shown and at least one product is displayed

3. WHAT HAPPENS BEHIND THE SCENES ?

	AI STEP	MCP + PLAYWRIGHT ACTION
1	Open the application	`navigate(url)`
2	Enter username	`locator.fill("#Email", "testuser")`
3	Enter password	`locator.fill("#Password", "Test@123")`
4	Click Login	`locator.click("#login")`
5	Search for product	`locator.fill("#small-searchterms", "laptop")`
6	Click Search	`locator.press("Enter")` or `locator.click("#small-search-box-button")`
7	Verify result	`locator.isVisible(".product-item")` OR `expect(locator.count(".product-item")).toBeGreaterThan(0)`

4. SAMPLE NATURAL LANGUAGE INSTRUCTION (TO AI AGENT)

"Open https://demowebshop.tricentis.com, login with username testuser and password Test@123. Search for laptop in the search box and verify that at least one product is displayed in the results."

5. WHAT YOU GET AS OUTPUT

- ✓ Steps executed by Playwright through MCP
- ✓ Page state after each step
- ✓ Screenshots (if enabled)
- ✓ Final verification result (Pass / Fail)
- ✓ Logs and actions performed

6. BENEFITS

- ✓ No need to write Playwright code manually for basic flows.
- ✓ Testers can automate using plain English instructions.
- ✓ AI understands the goal and handles step execution.
- ✓ Faster test creation and easy maintenance.
- ✓ Works with real browser, real data and real behavior.

✳ KEY TAKEAWAY: With Playwright MCP, you just tell the AI what to do in natural language, AI chooses the right tools, Playwright performs actions in the browser and you get the result.

@aman_views

7. AI-POWERED TEST CREATION & TEST GENERATION

AI can create tests, test data and Playwright scripts from requirements, manual test cases or user stories.

1. AI TEST GENERATION FLOW (HIGH LEVEL)

INPUT
Requirement / User Story / Manual Test Case

→

AI ANALYSIS
AI understands the requirement and identifies scenarios

→

GENERATE OUTPUT
- Test Cases
- Test Data
- Test Steps
- Assertions

→

PLAYWRIGHT SCRIPT
AI converts the test steps into Playwright code

→

EXECUTION
Run tests using Playwright MCP and get results

2. WHAT AI CAN GENERATE ?

- → Test cases (positive & negative scenarios)
- → Boundary and edge cases
- → Assertions / validations
- → Test data (valid, invalid, edge)
- → Page Object Model (POM) structure
- → Reusable Playwright functions
- → Setup / teardown steps
- → Locators using best practices
- → Waits and synchronization
- → Different browsers / viewports
- → API test cases (if required)
- → End-to-end user flows
- → Smoke / regression test suites

3. EXAMPLE : REQUIREMENT → TEST

REQUIREMENT :
As a user, I should be able to login, search for a product and add it to cart.

↓

AI GENERATES :
- Test Scenarios
- Test Steps
- Test Data
- Assertions
- Playwright Code

4. EXAMPLE OUTPUT (SAMPLE TEST SCENARIOS)

TEST ID	SCENARIO	TYPE	DESCRIPTION
TC_01	Valid Login	Positive	Verify user can login with valid credentials.
TC_02	Invalid Login	Negative	Verify error message with invalid credentials.
TC_03	Search Product	Positive	Search for a product and verify results.
TC_04	Add to Cart	Positive	Add a product to cart and verify cart count.
TC_05	Empty Search	Negative	Search with empty text and verify validation message.

5. SAMPLE PLAYWRIGHT CODE GENERATED BY AI

```
import { test, expect } from "@playwright/test";
test("Login and search product", async ({ page }) => {
    // Navigate
    await page.goto("https://demowebshop.tricentis.com");
    // Login
    await page.fill("#Email", "testuser");
    await page.fill("#Password", "Test@123");
    await page.click("text=Log in");
    await expect(page.locator("text=My account")).toBeVisible();
    // Search
    await page.fill("#small-searchterms", "laptop");
    await page.click("#small-search-box-button");
    await expect(page.locator(".product-item")).toHaveCountGreaterThan(0);
});
```

6. HUMAN REVIEW IS IMPORTANT

- ✓ Always review AI generated tests.
- ✓ Validate scenarios and assertions.
- ✓ Check locators and test data.
- ✓ Ensure coverage and edge cases.
- ✓ Do not blindly trust AI output.
- ✓ Maintain and optimize tests regularly.

7. BENEFITS

- → Faster test creation
- → Better coverage
- → Consistent test patterns
- → Reusable components
- → Reduces manual effort

✳ KEY TAKEAWAY : AI helps you create complete Playwright tests from plain English requirements. But human review and validation makes the tests reliable and effective.

@aman_views

8. PLAYWRIGHT MCP TOOLS & CAPABILITIES (DEEP DIVE)

Playwright MCP Server exposes Playwright capabilities as tools.
AI can discover these tools and use them to control the browser.

1. MAIN TOOL CATEGORIES

BROWSER CONTROL
- Launch Browser
- New Context
- New Page
- Close Page
- Close Browser

PAGE NAVIGATION
- Go To URL
- Reload Page
- Go Back
- Go Forward
- Navigate History

ELEMENT ACTIONS
- Click
- Double Click
- Type / Fill
- Select Option
- Check / Uncheck

ELEMENT LOCATORS
- By Text
- By Role
- By Label
- By CSS
- By XPath
- By Test ID

ASSERTIONS & VALIDATIONS
- Is Visible
- Is Hidden
- Text Contains
- Count Check
- Value Check

DATA & STATE HANDLING
- Get Text
- Get Attribute
- Get Value
- Get Count
- Get URL

UTILITIES
- Wait For
- Screenshot
- Evaluate JS
- Handle Dialog
- Cookies / Storage

AI DISCOVERS → SELECTS → EXECUTES → GETS RESULT

2. EXAMPLE TOOL CALLS

TOOL NAME	PURPOSE	EXAMPLE CALL (PARAMETERS)	EXAMPLE RESPONSE
browser.new_context	Create new browser context	{ "viewport": {"width":1280,"height":720} }	{"contextId": "ctx_1"}
browser.new_page	Open new page	{ "contextId": "ctx_1" }	{"pageId": "page_1"}
page.goto	Navigate to URL	{ "url": "https://demo.com" }	{"status": "loaded", "url": "https://demo.com"}
page.locator	Find element	{ "selector": "text=Login" }	{"elementId": "el_123"}
element.click	Click on element	{ "elementId": "el_123" }	{"success": true}
page.fill	Fill input	{ "selector": "#email", "value": "test@123.com"}	{"success": true}
page.is_visible	Check visibility	{ "selector": ".product-item" }	{"visible": true}
page.screenshot	Take screenshot	{ "path": "screenshot.png", "fullPage": true}	{"success": true, "path": "screenshot.png"}

3. LOCATOR STRATEGIES (PRIORITY ORDER)

1	Role Locator Best for accessibility.	getByRole("button", { name: "Login"})
2	Label Locator Best for form fields.	getByLabel("Email")
3	Text Locator Best for visible text.	getByText("Login")
4	Test ID Locator Best for stability.	getByTestId("login-btn")
5	CSS / XPath Use as last option.	locator("#login-btn") locator("//div[@id='login']")

4. COMMON ASSERTIONS

✓ Element is visible	expect(locator).toBeVisible()
✓ Element is hidden	expect(locator).toBeHidden()
✓ Text contains	expect(locator).toContainText("Welcome")
✓ Attribute has value	expect(locator).toHaveAttribute("href","/home")
✓ Count of elements	expect(locator).toHaveCount(3)
✓ Value check	expect(locator).toHaveValue("test@123.com")

5. WAITING STRATEGIES

- waitForSelector(selector, { state: "visible" })
- waitForTimeout(ms)
- waitForLoadState("domcontentloaded" / "load")
- waitForURL("**/dashboard")
- waitForResponse(urlPattern)

6. BEST PRACTICES

- ✓ Use Role / Label / Text locators first.
- ✓ Avoid hard waits, use auto waiting.
- ✓ Keep selectors stable and unique.
- ✓ Take screenshot on failure.
- ✓ Close page and context after test.

✳ KEY TAKEAWAY: Playwright MCP exposes rich tools for browser control, actions, validations and data handling. AI discovers the right tool, executes it and gets the result.

9. ADVANCED MCP FEATURES (DEEP DIVE)

Playwright MCP has some advanced features that make automation smarter, reliable and agent-friendly.

1. MULTI-STEP WORKFLOWS

AI can chain multiple tool calls in sequence to complete complex tasks.

Example Flow :

1. Go to URL
2. Login
3. Search Product
4. Add to Cart
5. Checkout
6. Verify Order

AI decides the next step based on current result.

2. CONDITION BASED DECISIONS

AI can make decisions based on page state or result.

Example :

- If login is successful → go to dashboard
- If error message → retry or show error
- If element not found → try alternative locator

3. ERROR HANDLING & RETRY

- ✓ AI can handle errors and retry intelligently.
- ✓ If a tool call fails, AI can try different approach.
- ✓ Helps in unstable networks and dynamic pages.

4. RESPONSE FORMAT (TYPICAL MCP RESPONSE)

```
{
  "tool": "page.goto",                          → Which tool was called
  "status": "success",                          → success / error
  "data": {                                     → Actual data returned
    "url": "https://demo.com",
    "title": "Demo Shop",
    "loadState": "domcontentloaded"
  },
  "metadata": {
    "timestamp": "2025-01-01T12:30:45Z",        → Extra info about call
    "durationMs": 1256                            (time, duration, etc.)
  }
}
```

5. SCREENSHOTS & SESSIONS

- AI can capture screenshot at any step.
- Useful for debugging and reporting.
- Screenshots are returned as base64 or URL.

Example Response :

```
{
  "tool": "page.screenshot",
  "status": "success",
  "data": {
    "path": "screenshot.png",
    "fullPage": true
  }
}
```

6. COMMON DATA & STATE HANDLING TOOLS

TOOL	PURPOSE	EXAMPLE CALL	EXAMPLE RESPONSE
page.evaluate	Run JS in page	{ "script": "document.title" }	{ "result": "Demo Shop" }
page.wait_for_selector	Wait for element	{ "selector": "#login-btn" }	{ "status": "success" }
page.get_cookies	Get cookies	{ }	{ "cookies": [{...}, {...}] }
page.set_cookies	Set cookies	{ "cookies": [{...}] }	{ "status": "success" }
browser.new_context	Create new context	{ "viewport": { "width":1280, "height":720}}	{ "contextId": "ctx_2" }
browser.close	Close browser	{ }	{ "status": "closed" }

7. MEMORY & CONTEXT MANAGEMENT

- ✓ AI remembers previous steps and results in the conversation.
- ✓ Maintains context like URLs, logged in user, cart items, etc.
- ✓ Helps in long workflows without losing state.

Example Use :

- Remember username after login
- Remember product ID after search
- Remember cart total during checkout

8. ADVANCED TIPS

- ✳ Use stable locators (testid, role, label) for reliability.
- ✳ Always wait for elements before interacting.
- ✳ Handle popups, dialogs and alerts.
- ✳ Use new context for isolation between tests.
- ✳ Take screenshots on failure for debugging.
- ✳ Keep workflows small and modular for better AI planning.

✳ KEY TAKEAWAY : These advanced features help AI Agents automate real-world scenarios in a reliable, intelligent and human-like way.

@aman_views

10. BEST PRACTICES & REAL WORLD USAGE OF PLAYWRIGHT MCP

Following best practices will help you build reliable, maintainable and secure MCP automations that work in real world scenarios.

1. BEST PRACTICES

- ✓ Give clear, specific and complete instructions.
- ✓ Break complex tasks into smaller steps.
- ✓ Use explicit locators (role, label, text, testid) instead of generic ones.
- ✓ Wait for elements and network properly.
- ✓ Validate each important step.
- ✓ Take screenshots on key steps and failures.
- ✓ Handle errors and retries smartly.
- ✓ Keep tests independent and idempotent.
- ✓ Clean up state after test (logout, clear cart, etc.).

2. REAL WORLD USE CASES

- End-to-End User Journeys
  Login → Search → Add to Cart → Checkout → Logout
- Regression Test Automation
  Convert manual test cases into automated tests.
- Smoke / Sanity Tests
  Verify critical flows after each build or deployment.
- Data Driven Testing
  Run same flow with multiple inputs.
- UI Monitoring / Validation
  Check UI changes, text, counts, visibility.
- AI QA Agents / Chatbots
  AI agent automates testing based on user queries.

3. SAMPLE REAL WORLD FLOW (E-COMMERCE)

1	Open Website	Navigate to the e-commerce site.
2	Login	Login with valid credentials.
3	Search Product	Search for a product.
4	Add to Cart	Add product to cart.
5	Checkout	Go to cart and place order.
6	Verify Order	Verify order confirmation.

4. HANDLING FAILURES & RETRIES

If a step fails:
- Capture screenshot
- Log error
- Retry the step (limited times)
- If still fails → Stop and report

Example Retry Logic (Conceptual):

```
for (let i = 1; i <= 3; i++) {
   try {
      // perform action
      break; // success
   } catch (err) {
      if (i === 3) throw err; // final failure
      // wait and retry
   }
}
```

5. SAMPLE MCP SCRIPT (E-COMMERCE FLOW)

```
{
  "mcp": "1.0",
  "task": "E-commerce checkout flow",
  "steps": [
    { "tool": "page.goto", "params": { "url": "https://demowebshop.tricentis.com" } },
    { "tool": "page.fill", "params": { "selector": "#Email", "value": "testuser" } },
    { "tool": "page.fill", "params": { "selector": "#Password", "value": "Test@123" } },
    { "tool": "page.click", "params": { "selector": "#login-button" } },
    { "tool": "page.fill", "params": { "selector": "#small-searchterms", "value": "laptop" } },
    { "tool": "page.click", "params": { "selector": "#small-search-box-button" } },
    { "tool": "page.click", "params": { "selector": ".product-item .add-to-cart-button" } },
    { "tool": "page.click", "params": { "selector": "#topcartlink" } },
    { "tool": "page.click", "params": { "selector": "#checkout" } },
    { "tool": "page.fill", "params": { "selector": "#billing-new-address-firstname", "value": "Aman" } },
    { "tool": "page.click", "params": { "selector": "#billing-buttons-container button" } },
    { "tool": "page.click", "params": { "selector": "#payment-method-buttons-container button" } },
    { "tool": "page.click", "params": { "selector": "#payment-info-buttons-container button" } },
    { "tool": "page.click", "params": { "selector": "#confirm-order-buttons-container button" } }
  ],
  "validate": [
    { "tool": "assert text", "params": { "selector": ".order-completed", "text": "Your order has been successfully processed!" } }
  ]
}
```

6. SECURITY & SAFETY TIPS

- ✓ Do not send MCP access to untrusted clients.
- ✓ Validate all inputs before executing.
- ✓ Avoid executing destructive actions (delete, transfer, payments) without confirmation.
- ✓ Mask sensitive data in logs.
- ✓ Use least-privilege where possible.

7. LIMITATIONS (CURRENT)

- Depends on Playwright capabilities.
- Cannot access system files or OS directly.
- AI may sometimes choose wrong locators.
- Complex captchas / anti-bot may block.
- Requires stable internet between client and MCP server.

✳ KEY TAKEAWAY: Follow best practices, write clear instructions and validate every step. This makes your Playwright MCP automation powerful, reliable and production-ready.

11. SAMPLE PROJECT STRUCTURE, SETUP & TIPS

A good project structure and setup makes your Playwright MCP project clean, scalable and easy to maintain.

1. SAMPLE PROJECT STRUCTURE

```
playwright-mcp-ai-agent/
├── mcp-server.js          ← Main MCP Server
├── tools/                 ← All MCP tool definitions
│   ├── browser-tools.js
│   ├── navigation-tools.js
│   ├── assertion-tools.js
│   ├── data-tools.js
│   └── utility-tools.js
├── test-data/             ← Test data, users, products etc.
│   ├── users.json
│   └── product.json
├── tests/                 ← AI generated / manual tests
│   └── sample-tests.spec.js
├── screenshots/           ← Screenshots from test runs
├── logs/                  ← Logs (optional)
├── .env                   ← Environment variables
├── package.json           ← Dependencies & scripts
└── README.md              ← Project info
```

2. SETUP STEPS

1. Create a folder for your project.
2. Initialize Node project: `npm init -y`
3. Install Playwright: `npm install playwright`
4. Install MCP SDK (example):
   `npm install @modelcontextprotocol/sdk`
5. Install dotenv (for .env): `npm install dotenv`
6. Run Playwright install (browsers):
   `npx playwright install`
7. Create MCP server and tools.
8. Add .env for any configuration.
9. Start MCP server and test your tools.

3. SAMPLE .env FILE

```
BASE_URL = https://demowebshop.tricentis.com
HEADLESS = false
DEFAULT_TIMEOUT = 30000
SCREENSHOT_PATH = ./screenshots
LOG_LEVEL = info
```

4. START MCP SERVER

```
# Start the MCP server

node mcp-server.js

# Server will be available for AI Agent
# (using MCP protocol)
```

5. PACKAGE.JSON (SAMPLE)

```
{
  "name": "playwright-mcp-ai-agent",
  "version": "1.0.0",
  "description": "AI driven Playwright automation using MCP",
  "main": "mcp-server.js",
  "scripts": {
    "start": "node mcp-server.js",
    "test": "node tests/sample-tests.spec.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "^0.5.0",
    "dotenv": "^16.4.0",
    "playwright": "^1.44.0"
  }
}
```

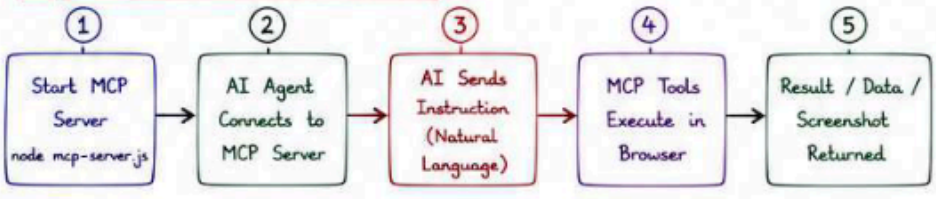
6. USEFUL COMMANDS

- Install browsers:
  `npx playwright install`
- Run tests:
  `npm test`
- Run in headed mode:
  `HEADLESS=false npm start`
- View trace (if enabled):
  `npx playwright show-trace trace.zip`
- View last screenshot:
  `open screenshots/last.png` (Mac) or `start screenshots\last.png` (Win)

7. TIPS FOR BETTER PROJECT

- ✓ Keep tools modular (one file = one category).
- ✓ Use meaningful tool names and descriptions.
- ✓ Handle errors and timeouts properly.
- ✓ Use explicit waits and stable locators.
- ✓ Log important actions for easy debugging.
- ✓ Store test data in JSON files.
- ✓ Keep screenshots only for failures.
- ✓ Write reusable functions inside tools.

8. EXAMPLE: START + TEST FLOW



9. TROUBLESHOOTING COMMON ISSUES

ISSUE	POSSIBLE CAUSE	SOLUTION
MCP server not starting	Port in use / wrong command	Check port, run: node mcp-server.js
Tool not found	Tool not registered	Check tool name and registration in server
Timeout errors	Slow page / bad locator	Increase timeout, use better locator, wait properly
Screenshots not saved	Wrong path / no permission	Check path in .env and folder permissions
AI gives wrong instruction	Unclear prompt	Give clear, specific instruction

10. FINAL CHECKLIST

- ✱ MCP server running
- ✱ All tools working
- ✱ Browsers installed
- ✱ Test data available
- ✱ Screenshots & logs working
- ✱ AI able to perform flows end-to-end

✱ KEY TAKEAWAY: Well structured project + good setup + best practices = Reliable, maintainable and scalable Playwright MCP automation.

12. FINAL SUMMARY & CHEAT SHEET

Playwright MCP + AI = Smarter, Faster and Reliable Test Automation.
Use AI to generate, Playwright MCP to execute and get real results.

1. WHAT WE LEARNED

- What is Playwright MCP and how it works.
- Architecture and complete flow.
- Installation and setup.
- Core tools and capabilities.
- AI powered test creation and test generation.
- Advanced features and real world usage.
- Best practices, project structure and tips.

2. MCP CORE BENEFITS

- ✳ Standardized protocol to control browser.
- ✳ AI can discover tools and use them automatically.
- ✳ Richer context, better decisions and results.
- ✳ Reduces manual effort and speeds up testing.
- ✳ Works across browsers and platforms.
- ✳ Enables intelligent, agent friendly automation.

3. MCP TOOLS QUICK REFERENCE

CATEGORY	TOOLS (EXAMPLES)	PURPOSE	EXAMPLE USE
Browser Control	new_context, new_page, close_page, close_browser	Manage browser, contexts and pages	Open browser, open page, close page/browser
Navigation	goto, go_back, go_forward, reload, navigate_history	Navigate to URLs and history	Go to page, back, forward, reload
Element Actions	click, fill, select_option, check, uncheck, dblclick	Interact with elements	Click button, fill input, select option
Locators	by_text, by_role, by_label, by_css, by_xpath, by_test_id	Find elements using different strategies	Find by text, role, label, css etc.
Assertions	is_visible, is_hidden, contains_text, count, value_check	Validate conditions and values	Check visibility, text, count, value
Data & State	get_text, get_attribute, get_value, get_count, get_url	Get data and state information	Get text, attribute, value, url etc.
Utilities	wait_for_selector, screenshot, evaluate_js, cookies	Wait, capture, run JS, handle cookies	Wait element, take screenshot, run JS, get cookies

4. AI CAN GENERATE

- ✓ Test scenarios from requirements.
- ✓ Test steps and Playwright scripts.
- ✓ Assertions and validations.
- ✓ Test data and locators.
- ✓ End to end user flows.
- ✓ Smoke / regression test suites.

5. BEST PRACTICES (QUICK REMINDER)

- ✓ Use stable locators (role, label, text, testid).
- ✓ Wait for elements and network properly.
- ✓ Break complex tasks into small steps.
- ✓ Handle errors and retries intelligently.
- ✓ Keep tests independent and idempotent.
- ✓ Take screenshots on failure.
- ✓ Clean up state after tests.

6. PROJECT CHECKLIST

- ✓ MCP server running
- ✓ All tools working
- ✓ Browsers installed
- ✓ Environment configured (.env)
- ✓ Sample tests passing
- ✓ Screenshots & logs working
- ✓ AI flows tested end-to-end

7. COMMON COMMANDS (QUICK RECALL)

```
→ npx playwright install                  # Install browsers
→ node mcp-server.js                      # Start MCP server
→ npm test                                # Run tests
→ HEADLESS=false npm start                # Run with UI
→ npx playwright show-trace trace.zip     # Open trace
→ open screenshots/last.png               # Open last screenshot
```

✳ KEY TAKEAWAY: Playwright MCP gives AI the power to control the browser.
AI understands, decides and executes using MCP tools.
Combine human planning + AI intelligence + MCP execution = Next level automation.

✳ KEEP PRACTICING, KEEP AUTOMATING, KEEP BUILDING! 🚀 ✳